From Bayesian Optimization to an Agentic, Claude-Driven Tuning Loop

Skipper-CCD operating-point optimization with an LTA readout

version 2: logbook, GP vs. Claude, and six parameters

Juan Cruz Estrada Vigil

October 14, 2026

Outline

1. The starting point: Gaussian-process (GP) optimization in Python
2. The agentic version: three cooperating agents
3. Replacing the GP with Claude
4. How Claude thinks and how it converges
5. The lab logbook
6. GP vs. Claude: same amplifier, same session
7. Six parameters: tuning the horizontal clock levels
8. Lessons and next steps

The problem

- Skipper-CCD read out with an **LTA** controller
- Find the operating parameters that give the best images
- Each evaluation is a **real measurement**: expose (LED) $\rightarrow$ set parameters $\rightarrow$ read out $\rightarrow$ score
- Expensive, noisy, black box $\Rightarrow$ Bayesian optimization

Parameter	Range	
V_{dd}	$[-23, -10]$	V
V_{drain}	$[-23, -10]$	V
V_r	$[-8, -5]$	V
`delay_H_overlap`	$[10, 30]$	clk

Optimizer works in $[-1, 1]^4$; values mapped to physical units.

The objective function

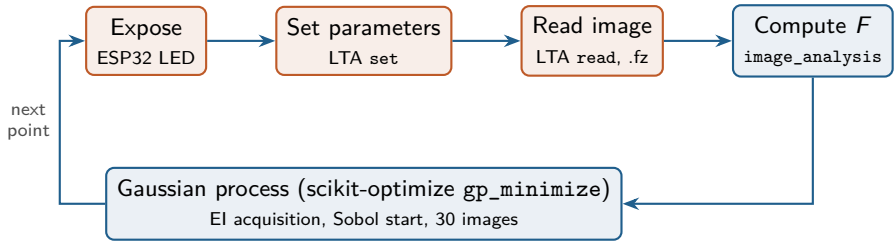
From each image, an active (exposed) region and an overscan region:

$$\text{gain} = \text{median}(\text{active}) - \text{median}(\text{overscan}), \qquad \sigma_{os} = \text{std}(\text{overscan})$$

$$F = \min\left(\left|\frac{\sigma_{os}}{\text{gain}}\right|,\ 1000\right) + \sum_i c_i\, \text{penalty}_i^{p_i} \qquad \text{(lower is better)}$$

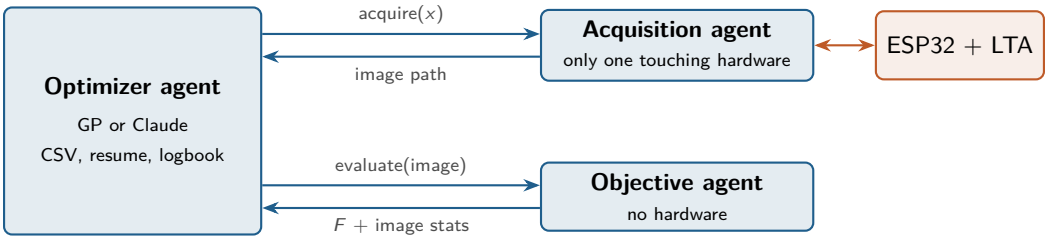
- "gain" is the signal level: with a fixed exposure it scales with the amplifier gain, so F tracks the read noise in units of the signal
- Optional penalties (negative gain, saturation, ...) exist but are all off
- Note the $|\cdot|$: an **inverted** signal (negative gain) scores like a normal one — this matters later

The original GP code



- `optimize_sensor_LTA.py` plus three modules: `lta_control` (hardware), `image_analysis` (FITS + F), `bo_core` (optimizer, CSV, plots)
- One JSON config per campaign; warm restart from a previous run
- These four files are kept unchanged throughout this work

Step 1: three cooperating agents



- Each agent is its own process (Python `multiprocessing`, standard library only)
- Messages are plain JSON dicts, all logged to `<run>_messages.jsonl`
- New driver `optimize_agents.py`: same config, options and output files

Messages between agents

```
{"id": 7, "kind": "acquire", "sender": "optimizer",
 "payload": {"iteration": 3, "x_norm": [0.12, -0.40, 0.75, -0.05]}}

{"id": 8, "kind": "reply", "sender": "acquisition", "reply_to": 7, "ok": true,
 "payload": {"image_path": ".../optimize_76.fz"}}

{"id": 9, "kind": "evaluate", "sender": "optimizer",
 "payload": {"iteration": 3, "image_path": ".../optimize_76.fz"}}

{"id": 10, "kind": "reply", "sender": "objective", "reply_to": 9, "ok": true,
 "payload": {"F": 0.0296, "stats": {"noise_overscan": 636.4,
             "charge_overscan": 5310.0, "charge_active": 26812.5, "gain": 21502.5}}}
```

- The image statistics travel with every score: the GP ignores them, Claude uses them
- An agent error comes back as `"ok": false` with its traceback; the run stops cleanly

Validation: identical to the original

Simulated hardware for testing:

- fake `lta.sh`: synthetic images whose noise and signal depend on the voltages
- fake ESP32 and FITS reader, fake Claude

Original script vs. agent version, same config and seed:

- identical `gp_results.csv`, progress CSV and images
- identical results on warm restart

Test suite

29 automated tests, all passing

equivalence, messages, resume, failures, Claude optimizer, retries, cost, logbook

No hardware, network or API key needed:

```
python3 -m pytest tests
```

Running in the lab: the surrounding software matters

Symptom	Cause	Fix
"No .fz image found"	relative image path: Python and the LTA daemon resolved it from different directories	absolute `output_base` in the config
Amplifiers mislabelled	config `amplifier` is the FITS HDU index; HDU 0 is empty	amplifier N (0–3) = `"amplifier": N+1`; recorded in every logbook entry
Claude run stopped after 16 images (`529 Overloaded`)	temporary API load	retries for up to $\sim$15 min
Interrupted runs could not be resumed	resume file written only at the end	`gp_results.csv` written after every image

Identifying the amplifier afterwards: recompute F for every FITS extension of one image; only one reproduces the logged value.

Step 2: Claude replaces the Gaussian process

1. **Sobol start**: 8 space-filling points (as for the GP)
2. **Each further point**: Claude (Opus 5.5) gets
   - parameter descriptions, bounds, objective definition
   - *all* measurements so far, in physical units
   - the image statistics (σ_{os}, σ_{act}, charges, gain), which the GP never sees
   - which amplifier is in use, operator notes, the logbook
3. Claude returns the next point as **structured JSON**; values clipped to the bounds

Drop-in replacement

`claude_minimize()` has the interface of `gp_minimize()` and returns the same result object: CSV, resume, pickle and plots unchanged. Switching is one line in the config.

```
"optimizer": {
  "type": "claude",
  "n_calls": 30,
  "n_initial_points": 8,
  "model": "claude-opus-5-5",
  "effort": "high" }
```

What Claude sees on each turn

```
Measurements so far (18):
# | Vdd | Vdrain | Vr | delay_H_overlap | F | noise_overscan | noise_active | charge_overscan |
     charge_active | gain
1 | -12.0 | -20.7 | -7.8 | 17.0 | 2.7763 | 869 | 920.6 | -1104 | -791 | 313
...
8 | -10.3 | -12.5 | -5.2 | 15.0 | 15.987 | 879.3 | 898.6 | -485 | -540 | -55
9 | -14.5 | -18.8 | -6.3 | 21.0 | 0.04715 | 682.7 | 1769 | 3365 | 1.784e+04 | 1.448e+04
...
18 | -17.5 | -20.2 | -5.0 | 30.0 | 0.02422 | 685.8 | 2456 | 6107 | 3.442e+04 | 2.832e+04

Measurements left in this campaign, including the next one: 12.
Choose the next point.
```

Each iteration prints and logs (`<run>_claude_decisions.jsonl`): the proposed point, a **summary of Claude's thinking**, tokens and cost.

Real rows (Oct 1, amplifier 2). Claude is never asked to write out its reasoning in the answer (Opus 5.5 declines that as "reasoning extraction"); the explanation comes from the thinking summaries the API returns.

How Claude thinks (1): using memory, noise and bounds

Real excerpts of the per-image thinking summaries, Oct 1:

Re-checking a remembered best point (logbook)

"The notebook's recorded best is (-14.5,-18.8,-6.3,21), so I should measure that point directly to verify reproducibility and use it as a reference." (It gave $F = 0.047$, not the remembered 0.019.)

Reasoning about noise and a parameter at its bound

"The repeat of trial #18 showed noticeable noise (∼0.01 variation) [. . .] I decide exploring Vr is valuable since we're sitting at a boundary value – I'll test Vr around -5.6 to -6.0 [. . .] to confirm whether -5.0 is truly optimal or just unexplored territory."

Noticing that the setup changed (broken amplifier 0)

"These conditions look drastically different from before – noise is much higher, F values are now in the 1.6-3 range instead of 0.02-0.03 [. . .]"

How Claude thinks (2): learning the structure from scratch

Oct 2, GP vs. Claude comparison: Claude does *not* read the logbook.

Image 16: separating the parameters

"I notice Vdd being more negative seems to help, while Vdrain is fairly flat between -10 and -13, so I want to test an edge case further out, maybe Vdrain -14.5 or -15 [. . .]"

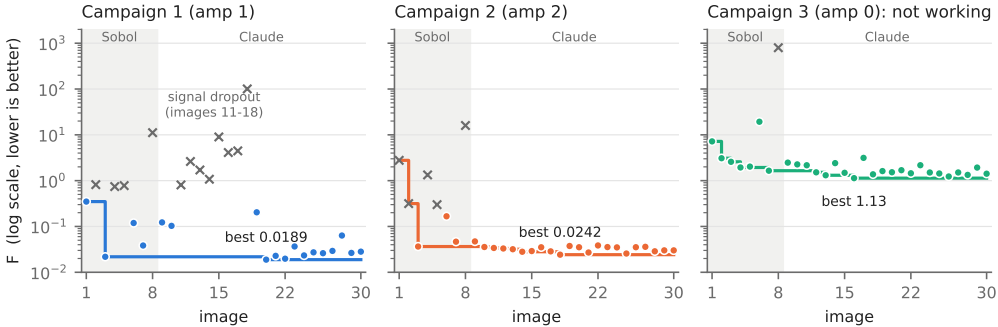
Image 17: a parameter at its bound

"The best region looks like Vdd=-23, Vr around -7.6/-7.7, delay 10, with Vdrain roughly flat between -10 and -15. I should explore further [. . .] since -23 seems to be at the boundary and already performing best."

Its end-of-campaign logbook entry

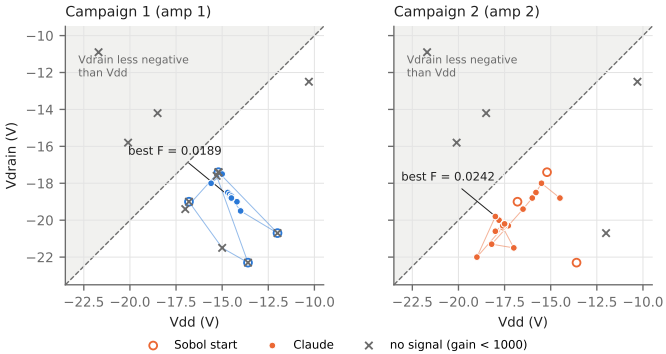
"Negative gain (#2, #3, #5, #7): charge_active was below charge_overscan, which is not a valid LED response. F uses |gain|, so these look deceptively moderate [. . .] Treat any gain < 0 as a failure."

How Claude converges: three amplifiers, Oct 1



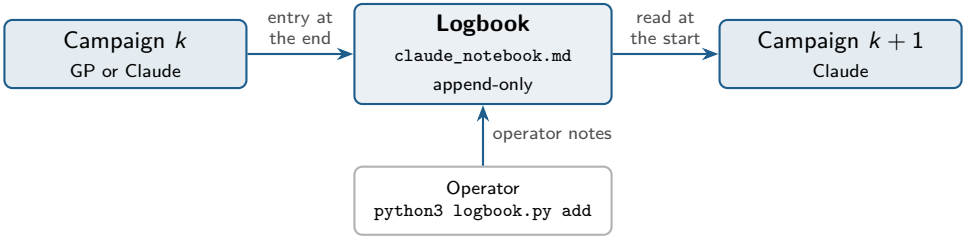
- Same 8 Sobol starting points, then 22 points chosen by Claude
- Converges within a few images after the start, then refines and re-measures
- Amplifier 0 was not working (σ_{os} 50× higher); Claude flagged it
- Oct 1 illumination was mostly light leaks (operator note, Oct 2): signal $\sim 2 \times 10^4$ ADU

What Claude found: the V_{dd}–V_{drain} relation (Oct 1)



- Shaded: V_{drain} less negative than V_{dd} $\Rightarrow$ negative gain
- Good points: V_{drain} 2.5–3 V more negative than V_{dd}
- Claude's conclusion in the logbook:
  "F is driven almost entirely by gain [. . .] most sensitive: the Vdd/Vdrain relationship"

The lab logbook: memory across campaigns



- Each API call starts from scratch: Claude does not learn between runs by itself
- **Every run** (GP, Claude, interrupted) appends an entry
- Claude reads it at the start as *prior knowledge*; current measurements take priority, and a later entry overrides an earlier one
- **Append-only**: entries are never changed; corrections are new entries

Anatomy of an entry (Oct 2, Claude run)

```
## 2026-10-02 17:46 | module skipper | amp 3 (HDU 4) | claude | 30 measurements (30 new) |
   best F = 0.42845 at Vdd=-23.0, Vdrain=-10.5, Vr=-7.4, delay_H_overlap=10.0

**Run facts** (recorded by the code)
- Optimizer: claude; config: config_compare_claude.json; images optimize_424.fz to
  optimize_453.fz; 17:36-17:46.
- Best F this run: 0.4284 at (Vdd=-23.0, Vdrain=-10.5, Vr=-7.4, delay_H_overlap=10.0), [...]
- Signal (gain, ADU): median 1177, max 1590; [...]
- Overscan noise (ADU): median 693, range 614-943.

**Best region.** The best results came from Vdd = -23.0, Vdrain = -10.5, Vr = -7.4 and
delay_H_overlap = 10. [...] Repeat scatter is about +-0.03-0.05.
**Sensitivity.** F is driven mainly by gain. Vdd had the largest effect [...]
**Suggestions for the next campaign.** [...] Both Vdd and delay are pinned at their bounds.
If hardware limits permit, extend Vdd below -23 and delay below 10 and test there first.
```

- **Header + run facts**: written by the code for every run (facts, not opinions)
- **Analysis**: written by Claude for Claude runs

Using the logbook

```
python3 logbook.py list           # one line per entry
python3 logbook.py show           # latest entry
python3 logbook.py show 3 5       # entries 3 and 5
python3 logbook.py search dropout
python3 logbook.py add --amp 3 -m "LED replaced"
python3 logbook.py add --amp 3    # write it in the editor
```

- Config: `"notebook_read": false` — Claude does not read it (fair comparisons) but the entry is still written
- `"notebook": false` — off completely

A real operator note (Oct 2)

"I think the optimization that we did yesterday 10/1/2026 were using as illumination a lot of light leaks on the system. Now I am using as illumination the LED, I closed most of the light leaks. I think that is why the "gain" [. . .] is much lower than before."

Marked `operator note`; Claude treats these as first-hand observations.

What the logbook caught

- **A broken amplifier** (Oct 1): *"Setup state changed sharply this campaign [...] noise_overscan was 3.1–4.5e4 ADU, about 50× [...] F values from this campaign are not comparable with earlier entries."*
- **Signal decay** (Oct 2, 15:15): gain halving image after image; *"When the reference gain starts to decay, stop optimizing and investigate the LED or exposure chain"*
- **Its own limits**: the shared logbook did not record the amplifier, so Claude could only guess ("hardware, offset or cabling"). Fixed: every entry now records it, and Claude is told which amplifier is in use
- **Illumination change**: the operator note on light leaks explains why the Oct 2 signal is $\sim$15× lower — and changed how we read later runs

Comparison setup

- Same session, back to back (Oct 2): GP 17:28–17:36, Claude 17:36–17:46
- Same amplifier: amp 3 (`--amplifier 4`), LED-only illumination
- Same budget: 30 images, 8 Sobol + 22 guided
- Same agents and objective; only the optimizer differs
- Claude does **not** read the logbook: no prior knowledge

Commands

```
python3 optimize_agents.py \
  --config config_compare_gp.json \
  --amplifier 4

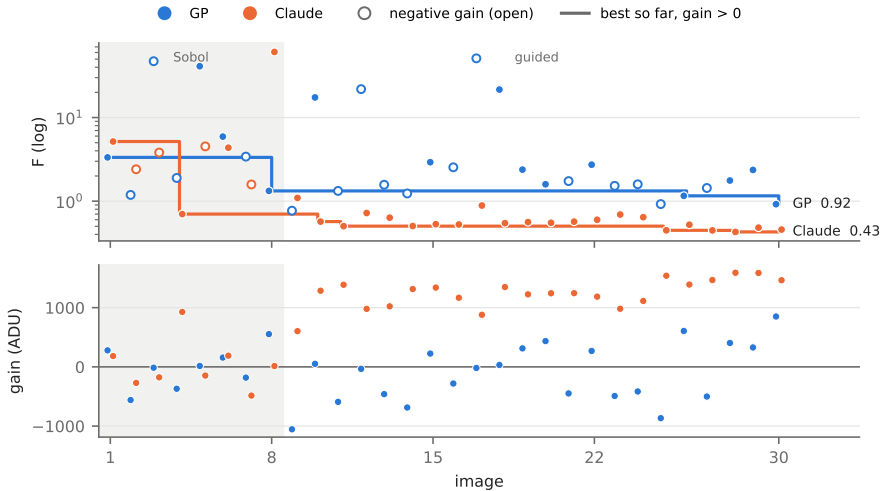
python3 optimize_agents.py \
  --config config_compare_claude.json \
  --amplifier 4
```

Results

	GP	Claude
Best F (any sign)	0.769 at $(-23, -23, -6.7, 30)$	**0.428** at $(-23, -10.5, -7.4, 10)$
Gain at that point	$-$**1054 ADU** (inverted)	$+1590$ ADU
Best F with gain > 0	0.920 (gain 850)	**0.428** (gain 1590)
Images with gain > 0	14 of 30 (10 of 22 guided)	26 of 30 (all 22 guided; 20 above 900 ADU)
Behaviour	guided points mostly at the bounds	region found by image 10, then refined
Repeatability at best point	—	6 images: $F = 0.43$–0.52
Time / cost	8 min / free	10 min / \$0.28

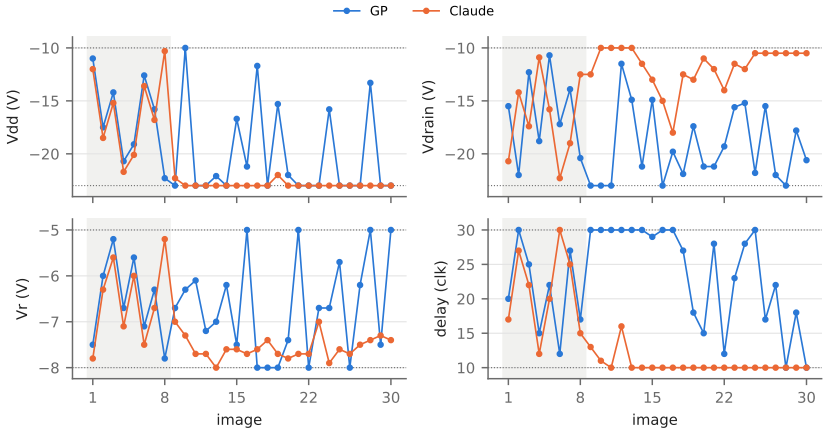
- Among valid (positive-gain) images Claude reaches $\sim 2\times$ lower F with $\sim 2\times$ more signal
- The GP's "best" image is an inverted signal: it optimized $|\text{gain}|$

Every image: score and signal



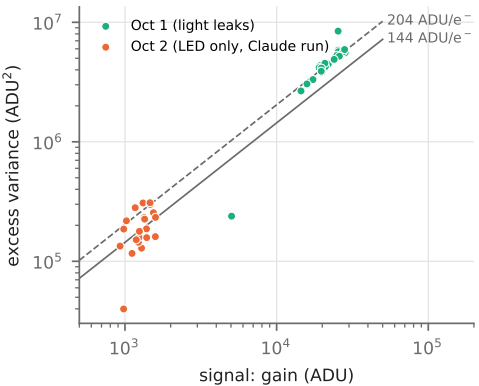
Top: F per image; line = best so far among positive-gain images. Bottom: signal (gain). Claude finds a region of consistent positive signal; the GP keeps jumping between positive and negative gain.

How each method moves through the parameter space



Dotted lines: bounds. Claude converges to $V_{dd} = -23$, $V_{drain} \approx -10.5$, $V_r \approx -7.5$, delay $= 10$ and then explores *one parameter at a time* around it; the GP keeps sampling the edges of the space.

Is the LED signal real? The shot-noise check



- Light adds photon shot noise: excess variance $\sigma^2_{act} - \sigma^2_{os} \approx k \cdot S$
- Oct 1 (light leaks): $k \approx 204$ ADU/e$^-$, $S \sim 2 \times 10^4$ ADU
- Oct 2 (LED only, Claude's region): $k \approx 144$ ADU/e$^-$, $S \sim 1.4 \times 10^3$ ADU
- Same order of $k \Rightarrow$ the small Oct 2 signal is real light, $\sim$10 e$^-$ per pixel — not an offset

Why the GP lost, and a second pair that agrees

- F uses |gain|: an inverted signal looks as good as a real one
  - the GP minimizes F blindly and ranked an inverted image first
  - Claude sees the statistics and concluded, unprompted: *"Treat any gain < 0 as a failure"*
- With weak signal, F is noisy and nearly flat over most of the space: hard for a GP with no notion of what a valid image is

Earlier pair, Oct 2	GP (16:09)	Claude (16:19)
Best F	0.878	**0.440**
at (V_{dd}, V_{drain}, V_r, delay)	(-16.5, -10.0, -5.0, 10)	(-19.5, -10.5, -7.4, 11)

Claude reached the same corner ($V_{drain} \approx -10.5$, $V_r \approx -7.5$, delay ≈ 10) in both runs, with $F = 0.44$ and 0.43.

Cost

Claude Opus 5.5 (list price)	per M tokens
input	$4.00
output (incl. thinking)	$20.00
cached input, read	$0.20

Campaign (22 decisions)	cost
Oct 1, amplifiers 1, 2, 0	$0.31, $0.31, $0.39
Oct 2, 16:19	$0.27
Oct 2, 17:36 (comparison)	$0.28
Oct 14, 6 param., 42 dec.	$0.78

List-price estimates; plus one logbook-entry request per campaign.

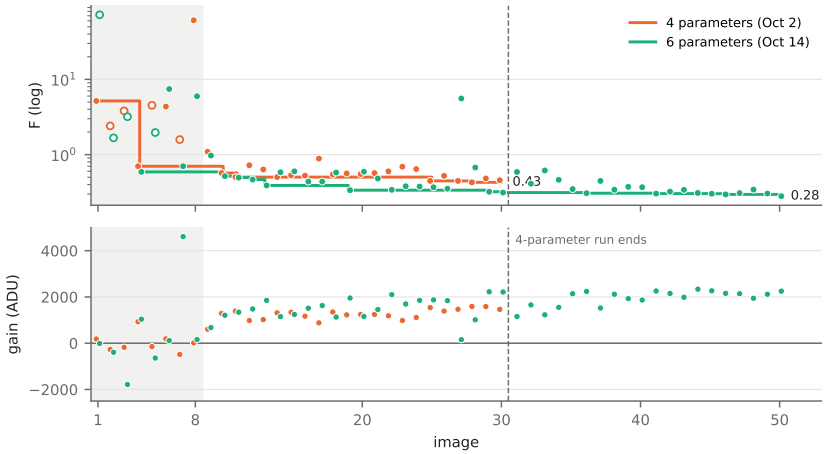
- Tracked automatically: tokens and cost per image, running total, summary
- ≈$0.01–0.02 per decision, ≈$0.3 per 10-minute campaign
- Small compared to the lab time per image

Adding the horizontal clock levels

- The LTA voltage script sets five horizontal clocks from two numbers: `hh` (high) and `hl` (low)
- New in the config: a *linked* parameter, one optimized value sent to every listed LTA variable
- Script defaults: $hh = -2\,\mathrm{V}$, $hl = -4.5\,\mathrm{V}$; bounds opened to $[-10, 10]\,\mathrm{V}$
- The script's ordering constraints ($hh > hl$, ...) are **not** enforced: the optimizer may try inverted clocks
- Same agents; only the config and the acquisition agent's expansion changed

```
{"name": "hh",
 "lta_var": ["h1ah","h1bh","h2ch",
             "h3ah","h3bh"],
 "bounds": [-10, 10],
 "precision": 1},
{"name": "hl",
 "lta_var": ["h1al","h1bl","h2cl",
             "h3al","h3bl"],
 "bounds": [-10, 10],
 "precision": 1}
```

4 vs. 6 parameters: Claude, amplifier 3, LED only



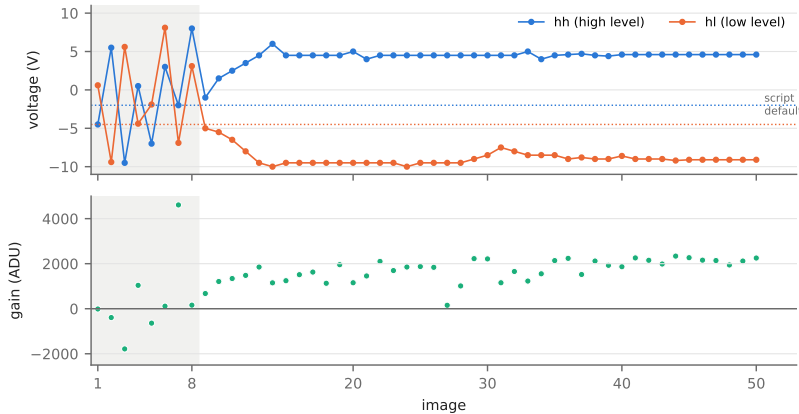
Line: best F so far among images with gain > 0; open circles: negative gain. Grey: Sobol start (8 images). Same light, same amplifier, Oct 2 vs. Oct 14.

Result: lower F through more signal

Claude, amplifier 3 (HDU 4)	4 parameters (Oct 2)	6 parameters (Oct 14)
Best F	0.428	**0.280**
Best F within 30 images	0.428	**0.313**
Last 10 images, median F	0.50	**0.31**
Gain at best (ADU)	1590	**2251**
Overscan noise at best (ADU)	681	631
Guided images with gain > 0	22 / 22	42 / 42
Best point (V_{dd}, V_{drain}, V_r, delay)	(-23, -10.5, -7.4, 10)	(-19.5, -10.0, -7.6, 10)
(hh, hl)	(-2, -4.5) fixed	($+4.6$, -9.1)
Images, time, cost	30, ∼10 min, \$0.28	50, 17 min, \$0.78

- Noise unchanged (∼650–690 ADU); the gain rose by ∼40% ⇒ F down ∼35%
- A plateau, not a lucky image: $F = 0.30$–0.35 for $hh \approx 4.5$, $hl \approx -8.5$ to -9.2
- Larger than the repeat scatter of F (± 0.03–0.05); earlier 4-parameter runs: 0.37–0.47

How Claude moved the clocks



After the Sobol start Claude opens the swing step by step: *hh* up to $\sim+4.5$, *hl* down to ~-9 (dotted: script defaults), and the gain follows.

What Claude was thinking

Image 9

"I'll favor the normal hh > hl ordering."

Image 11

"It had a bigger swing between hh and hl, which might be the key factor driving that gain difference."

Image 14

"Increasing the hh–hl swing seems to improve gain."

Image 45

"There's a sharp cliff around hh $\sim$ 4.7 where gain drops dramatically."

Thinking summaries from the decisions log; the hypothesis (swing $\rightarrow$ gain) was formed early and then tested.

Caveats

- **Inverted ordering**: $hh = +4.6\,\mathrm{V}$ is far outside the script's convention; inspect the images (charge transfer, trails, bad columns) before adopting it
- F **is not image quality**: it rewards gain/noise; a large swing may cost charge-transfer efficiency or spurious charge, which F does not see
- **Budget**: 50 vs. 30 images, but 6 parameters reached 0.313 already by image 30
- **One run each**: repeat the 6-parameter run (and a GP run on the same space)
- **Fakes are possible**: earlier narrow-bound runs (Oct 7) gave $F \approx 0.06$ from a huge negative gain and high overscan noise; Claude flagged them itself as artifacts
- **Crash fixed**: a fixed parameter (bounds $[8, 8]$) made Claude's runs divide by zero; fixed and tested

Gaussian process vs. Claude

	Gaussian process	Claude
Information used	parameters and F only	F, image statistics, notes, logbook
Validity of an image	unknown to it	reasons about it (e.g. negative gain)
Reproducible	yes (fixed seed)	no, but it re-measures to estimate noise
Explains choices	no	thinking summary per image, logbook entry
Memory across runs	warm restart (data only)	data + written lessons
Cost	free	$\sim$\$0.3 per campaign
Failure modes	outliers, flat/noisy F, inverted signals	API availability, safety filters

Same agents, same outputs: switching is one line in the config.

Lessons and next steps

- **Fix the objective for both**: reject or penalize gain ≤ 0 (the `negative_gain` penalty exists, currently off); the GP should not win with an inverted image
- **Extend the bounds**: V_{dd} and delay ended at their limits ($-23\,\mathrm{V}$, $10\,\mathrm{clk}$)
- **Repeat the comparison** with the corrected objective and new bounds, and on a second amplifier
- **Noise of** F: repeated images at a fixed point, to state which differences are real
- **Stopping rule**: today a fixed budget (30 or 50 images)
- **Six parameters**: inspect images at $hh = +4.6$, $hl = -9.1$; repeat the run; a GP run on the same 6-parameter space; check charge transfer at large swing
- **Better objective**: e.g. "best flat field for astronomy", with criteria drawn from the literature and reviewed before use

Summary

1. Started from a GP optimizer for the Skipper-CCD operating point
2. Rebuilt it as three agents (acquisition, objective, optimizer), verified identical
3. Replaced the GP by Claude as a drop-in: it uses the image statistics and explains every choice
4. Added a lab logbook: every run appends facts (and Claude's analysis); Claude reads it as memory; operators add notes; append-only
5. Same-session comparison on one amplifier: Claude found a consistent positive-signal region with $\sim 2\times$ lower F; the GP's best image was an inverted signal
6. Six parameters (adding the horizontal clock levels): Claude raised the gain $\sim 40\%$ at equal noise, F 0.43 $\rightarrow$ 0.28, on a stable plateau

Code: `optimize_agents.py`, `agents/`, `logbook.py`, `tests/`